

1
RU.17701729.05.03-01 ТЗ 01-1
АННОТАЦИЯ

Настоящий документ является пояснительной запиской к разработанному продукту - казуальной игре «Цветение».

В разделе «Введение» содержится название программы и сведения о техническом задании, на основе которого ведется разработка.

Раздел «Назначение и область применения» описывает функции, реализуемые программой, сферу использования и игровой контекст.

В разделе «Технические характеристики» содержатся следующие подразделы: постановка задачи на разработку программы, описание алгоритмов, используемых в программе, и аргументация сделанного выбора, характеристика и обоснование выбора состава технических и программных средств.

В разделе «Ожидаемые технико-экономические показатели» оценивается потенциальный спрос на продукт и его преимущества перед существующими отечественными и зарубежными аналогами.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

АННОТАЦИЯ	1
1 ВВЕДЕНИЕ	4
1.1 Наименование программы	4
1.2 Основание для разработки	4
2 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	5
2.1 Назначение приложения.....	5
2.1.1 Функциональное назначение	5
2.1.2 Эксплуатационное назначение	5
2.2 Краткая характеристика области применения	5
3 ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	6
3.1 Постановка задачи	6
3.1.1 Алгоритм процедурной генерации игровых уровней типа 1 (Алгоритм типа 1) 6	
3.1.1.1 Инициализация игровой сетки	6
3.1.1.2 Генерация уровня.....	6
3.1.1.3 Представление вершин графа.....	6
3.1.1.4 Отображение и проверка победы	7
3.1.2 Алгоритм процедурной генерации игровых уровней типа 2 (Алгоритм типа 2) 7	
3.1.2.1 Инициализация и генерация уровня	7
3.1.2.2 Генерация точек и построение графа.....	7
3.1.2.3 Расчет максимального потока.....	7
3.1.3 Регистрация и аутентификация пользователей.....	7
3.1.3.1 Хранение результатов игроков	8
3.1.3.2 Подсчет очков.....	8
3.1.3.3 Запрос рекордного времени и сохранение нового рекорда.....	8
3.2 Сравнительная характеристика	8
3.2.1 Обоснование выбора алгоритмов для уровня типа 1	8
3.2.2 Обоснование выбора алгоритмов для уровня типа 2	10
3.3 Описание и обоснование состава технических и программных средств.....	13
3.3.1 Состав технических и программных средств	13
3.3.2 Обоснование состава технических и программных средств.....	13
3.3.3 Описание и обоснование архитектуры программы	14
3.4 Описание алгоритма работы программы	14
4 ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ.....	16
4.1 Ориентировочная экономическая эффективность.....	16
4.2 Предполагаемая потребность	16

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.05.03-01 ТЗ 01-1

4.3 Экономические преимущества разработки по сравнению с лучшими отечественными и зарубежными образцами или аналогами.....	16
ПРИЛОЖЕНИЕ 1	18
ПРИЛОЖЕНИЕ 2	19
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	20
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ	22

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.1 Наименование программы

Наименование программы -«Казуальная игра “Цветение”».

Наименование программы на английском языке -«Casual game “Blooming”».

1.2 Основание для разработки

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1 Назначение приложения

2.1.1 Функциональное назначение

Игра «Цветение» является казуальной головоломкой. Ее задача - предоставить пользователю интуитивный и спокойный игровой процесс. Игра развивает логическое и пространственное мышление, а также навыки планирования.

2.1.2 Эксплуатационное назначение

Программа может эксплуатироваться при наличии мобильного телефона на базе ОС Android.

Конечные потребители - пользователи, без специальной подготовки, любители казуальных игр.

2.2 Краткая характеристика области применения

Программа представляет собой казуальную головоломку для мобильных устройств на базе Android, предназначенную для проведения досуга. Игра ориентирована на людей, предпочитающих медитативный игровой процесс с визуально приятной механикой, основанной на трансформации графовых структур.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3 ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1 Постановка задачи

Необходимо разработать казуальную игру, предоставляющую пользователю возможность регистрации и авторизации, прохождения игровых уровней двух типов, а также просмотра таблицы лидеров для каждого типа уровня, в которой фиксируются и отображаются результаты всех игроков

3.1.1 Алгоритм процедурной генерации игровых уровней типа 1 (Алгоритм типа 1)

3.1.1.1 Инициализация игровой сетки

Модуль должен формировать прямоугольную сетку вершин на основе заданных параметров (количество строк, столбцов и шаг между вершинами). Сетка включает вершины трех категорий:

- Вращаемые вершины, являющиеся частью пути
- Невращаемые вершины, являющиеся частью пути
- Граничные вершины, не входящие в путь

3.1.1.2 Генерация уровня

Модуль должен реализовывать полный цикл генерации уровня

- Построение сетки: последовательное формирование граничного ряда, чередующихся рядов с вращаемыми и промежуточными вершинами.
- Построение остовного дерева методом итеративного поиска в глубину. Начальная вершина выбирается случайным образом среди вращаемых. Количество вершин, которые необходимо обойти, рассчитывается по формуле: $3 \times rows \times cols - rows - cols$.
- Перемешивание графа путем применения операции вращения к случайно выбранным вращаемым вершинам. Количество итераций выбирается равномерно в диапазоне от $rows \times cols / 2$ до $rows \times cols$ включительно.

3.1.1.3 Представление вершин графа

Каждая вершина должна хранить:

- Мировые координаты.
- Множество смежных вершин
- Список соседей, упорядоченный по часовой стрелке на основе угла относительно начала координат (точки (0,0,0))
- Словарь ребер с индексами конечных вершин в упорядоченном списке.

Помимо хранения данных, вершина должна обеспечивать:

- Сортировку смежных вершин по часовой стрелке при первом добавлении ребра.
- Регистрацию ребер с сохранением индекса конечной вершины.
- Операцию вращения: для каждого исходящего ребра конечная вершина переключается на следующую в круговом списке соседей, после чего запускается проверка связности графа.
- Представление ребер графа

Каждое ребро должно хранить ссылки на начальную и конечную вершины, гарантируя, что начальная вершина всегда является вращаемой. Модуль должен обеспечивать операцию изменения конечной вершины ребра в результате поворота.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.1.1.4 Отображение и проверка победы

Модуль должен обеспечивать запуск логики завершения уровня и отображения результатов при обнаружении единой связной компоненты.

3.1.2 Алгоритм процедурной генерации игровых уровней типа 2 (Алгоритм типа 2)

3.1.2.1 Инициализация и генерация уровня

Модуль должен обеспечивать полный цикл генерации уровня на основе следующих параметров:

- количество вершин
- размеры игрового поля
- диапазон допустимых пропускных способностей ребер

Цикл генерации включает:

- Построение планарного графа на основе триангуляции Делоне
- Назначение ребрам случайных пропускных способностей в заданном диапазоне.
- Выбор множеств вершин-истоков и вершин-стоков.
- Расчет максимального потока, представляющего собой эталонный ответ головоломки

3.1.2.2 Генерация точек и построение графа

Необходимо обеспечить генерацию заданного количества точек при помощи квазислучайной последовательности Соболя с масштабированием в координаты игрового поля.

Использовать случайное скремблирование для нарушения детерминированности при каждом запуске и вычисление каждой следующей точки через операцию XOR над направляющими числами с нормировкой результата в диапазон $[0, 1]$.

Для построения графа необходимо использовать NuGet пакет DelaunatorSharp.

3.1.2.3 Расчет максимального потока

Модуль должен реализовывать алгоритм Диницы для нахождения максимального потока в ориентированном графе и возврат итогового значения максимального потока.

Кроме того, должен поддерживаться случай нескольких истоков и нескольких стоков:

- Создание суперистока, соединяемого с каждой вершиной-истоком ребром бесконечной пропускной способности.
- Создание суперстока, к которому подключается каждая вершина-сток аналогичным ребром.
- Сведение задачи к стандартной однопоточковой и запуск расчета максимального потока.

3.1.3 Регистрация и аутентификация пользователей

Модуль должен обеспечивать взаимодействие с сервисом аутентификации Firebase и включать следующие функции:

- Инициализацию сессии авторизации с подпиской на событие смены состояния для отслеживания входа и выхода пользователя в реальном времени.
- Асинхронную регистрацию нового пользователя по электронной почте и паролю с последующей установкой отображаемого имени профиля. В случае ошибки

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

исключение передается вызывающему коду.

- Асинхронный вход в существующую учетную запись по электронной почте и паролю с автоматическим обновлением состояния авторизации.
- Обработку смены состояния авторизации с фиксацией фактов входа и выхода из системы.
- Обновление отображаемого имени в профиле текущего пользователя.
- Инициирование отправки письма для сброса пароля на указанный адрес электронной почты с обработкой как специфических ошибок сервиса, так и общих системных исключений.

3.1.3.1 Хранение результатов игроков

- Модуль должен обеспечивать запись результатов в Firebase Realtime Database
- Инициализацию подключения к корневому узлу базы данных как точке входа для всех операций.
- Асинхронную запись итогового счета игрока в узел таблицы лидеров, используя уникальный идентификатор пользователя в качестве ключа записи, без блокировки основного потока.

3.1.3.2 Подсчет очков

Модуль должен вычислять итоговый счет игрока по формуле:

$$Score = (T_{opt} / T_{finish}) \times 1000,$$

где T_{opt} - рекордное время прохождения уровня в секундах, T_{finish} - фактическое время игрока в секундах.

3.1.3.3 Запрос рекордного времени и сохранение нового рекорда

Запрос рекордного времени из базы данных должен происходить по уникальному идентификатору текущего пользователя перед вычислением счета. Если запись отсутствует (первое прохождение), в качестве рекордного времени используется фактическое время игрока, что гарантирует 1000 очков за первое прохождение. При установлении нового рекорда обновленное время и соответствующий счет сохраняются в базу данных. Если рекорд не побит, запись не производится.

3.2 Сравнительная характеристика

3.2.1 Обоснование выбора алгоритмов для уровня типа 1

Таблица 1: Структура уровня

	Сетка из вершин и ребер	Триангуляция Делоне [16]
Описание	Фиксированная структура, обеспечивающая предсказуемое положение для каждой вершины и ребра	Структура дает большую вариативность уровней за счет нефиксированных положений для ребра. При этом алгоритм гарантирует, что ребра не будут пересекаться
Число уникальных графов ($\tau(G)$)	По матричной теореме Кирхгофа $\tau(G)$ для графов с n вершинами растет	В статье [15] приводится обоснование, число уникальных остовных деревьев $\sim \lim_{n \rightarrow \infty} e^{Cn} = \infty$, то есть алгоритм на каждом

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

	экспоненциально в зависимости от топологии	новом наборе точек будет выдавать дерево, не имеющее сходств созданными ранее
Временная сложность построения	Создаются $N = 3 \times rows_c \times cols_c + rows_c + cols_c$ вершин и $E = 3 \times rows_c \times cols_c - rows_c - cols_c - 1$ ребро. При этом $rows_c \times cols_c$ - общее число всех вершин на поле, а следовательно временная сложность составит $O(n)$, где n - число вершин	Официальный NuGet пакет C# в среднем строит триангуляцию за $O(n \times \log(n))$, где n - число вершин
Затраты по памяти	В памяти хранятся только вершины и ребра, без дополнительных структур. Сложность: $O(n)$, где n - число вершин	$O(n)$
Вывод	Таким образом, сетка из вершин и ребер является наиболее оптимальным вариантом с точки зрения производительности и вариативности графовых структур	

Таблица 2: Построение минимального остовного дерева

	DFS [28]	Алгоритм Прима [18]	Алгоритм Краскала [19]
Описание	Обход графа в глубину. Остовное дерево строится как дерево обхода - каждое ребро, по которому впервые посещается новая вершина, включается в дерево. Не гарантирует минимальности весов, но обеспечивает связность и уникальность структуры для каждого порядка обхода	Жадный алгоритм: начинает с произвольной вершины и на каждом шаге добавляет минимальное по весу ребро, соединяющее уже построенное дерево с непосещенной вершиной	Жадный алгоритм: сортирует все ребра по весу и последовательно добавляет минимальные ребра, не образующие цикл
Временная сложность	$O(n)$, где n - число вершин (для сетки, где $E \sim n$)	$O(n \log n)$ при реализации через бинарную кучу	$O(E \log E)$, где E - число ребер
Затраты по памяти	$O(n)$ - хранится стек вызовов и список посещенных вершин	$O(n)$ - очередь с приоритетом и массив ключей	$O(n + E)$ - хранение всех ребер для сортировки
Вывод	В силу простоты структуры и отсутствия на данном этапе весов у ребер был выбран DFS, так как он является наиболее оптимальным вариантом с точки зрения производительности: линейная сложность $O(n)$ по времени и памяти		

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Таблица 3: Алгоритмы генерации псевдослучайных чисел

	Криптографически стойкий генератор RandomNumberGenerator	Линейный конгруэнтный генератор System.Random
Описание	Криптографически стойкий генератор из пространства имен System.Security.Cryptography. Использует источники энтропии ОС (аппаратный шум, прерывания). Каждый вызов гарантирует непредсказуемость следующего числа даже при знании предыдущих	Стандартный генератор псевдослучайных чисел на основе линейного конгруэнтного метода: $x_{n+1} = (ax_n + c) \bmod m$. Детерминированная последовательность, полностью определяемая начальным seed
Равномерность распределения	Высокая - распределение статистически равномерно по всему диапазону, коллизии крайне редки	При плохом выборе параметров a, c, m возникает зависимость между последовательными числами
Воспроизводимость	Отсутствует намеренно - два запуска с одинаковыми параметрами дадут разные последовательности	Полная воспроизводимость при одинаковом seed - удобно для отладки и тестирования
Скорость генерации	O(1) на число, медленнее System.Random на 28% из-за обращения к ОС	O(1) на число, работает быстрее за счет простых арифметических операций
Вывод	RandomNumberGenerator является наиболее оптимальным вариантом с точки зрения уникальности генерируемых значений. Отсутствие воспроизводимости и обращение к энтропии ОС гарантируют, что каждый новый набор чисел не будет повторять предыдущие последовательности, что очень важно для генерации уникальных графов	

Оценка вычислительной сложности:

$$N = rows_c \times cols_c$$

Построение сетки вершин: O(N)

Построение остоного дерева O(N)

При сортировке для каждой вращаемой вершины упорядочивается не более четырех соседей, что соответствует O(1) на вершину, а значит временная сложность: O(N)

Итоговая временная сложность Алгоритма типа 1 составляет O(N)

3.2.2 Обоснование выбора алгоритмов для уровня типа 2

Таблица 4: Построение планарного графа

	Триангуляция Делоне [16]	Граф Габриэля [22]	k-NN граф [23]
Описание	Описано в разделе про Алгоритм типа 1	Подграф триангуляции Делоне: ребро между двумя вершинами включается только если	Для каждой вершины находятся k ближайших соседей, с которыми она соединяется ребрами. Топология

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

		окружность, построенная на этом ребре как на диаметре, не содержит других вершин	определяется параметром k и взаимным расположением точек. Граф может быть несвязным при малых k
Число уникальных графов $\tau(G)$		Поскольку граф является подграфом Делоне, число уникальных структур теоретически меньше, чем у полной триангуляции	Сильно зависит от k : при малых значениях граф может быть несвязным, при больших приближается к полному графу, что не дает гарантий, что уровень всегда будет визуально приятен
Временная сложность построения		$O(N \times \log N)$ - строится как граф триангуляции Делоне, но с дополнительной проверкой условия Габриэля для каждого ребра	$O(N \times k \times \log N)$ - для каждой из N вершин ищутся k ближайших соседей с использованием k -d дерева
Затраты по памяти		$O(N)$	$O(N \times k)$ - для каждой вершины хранится список из k соседей. При больших k существенно превышает $O(N)$
Вывод	Триангуляция Делоне является наиболее оптимальным вариантом: гарантирует связность и планарность графа, обеспечивает экспоненциально растущее число уникальных топологий при $O(N \times \log N)$ по времени и $O(N)$ по памяти		

Таблица 5: Алгоритм нахождения максимального потока в ориентированном графе.

	Алгоритм Диницы [24]	Алгоритм Форда-Фалкерсона [25]
Описание	Находит блокирующий поток через DFS с оптимизацией указателя, исключающей повторное рассмотрение уже обработанных ребер. За одну итерацию пропускает сразу несколько увеличивающих путей	Итеративно находит увеличивающие пути от истока к стоку в остаточном графе с помощью DFS и увеличивает поток вдоль них. Завершается, когда увеличивающий путь не найден
Худшее время работы	$O(E \times V^2)$, то есть $O(N^3)$, поскольку граф планарен	$O(F \times E)$ где F - значение максимального потока. При больших пропускных способностях ребер F может быть огромным, что делает алгоритм непригодным для графов с большими весами
Среднее время работы	$O(E \times V^2)$	Для планарного графа $O(N^3)$
Практическая скорость	Значительно быстрее на практике за счет снижения повторных обходов	Сильно деградирует на графах с большими пропускными способностями ребер, потому что число итераций

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

	обработанных ранее ребер	пропорционально F
Вывод	Алгоритм Диницы является оптимальным выбором так как время работы не зависит от значений пропускных способностей, а оптимизация по сравнению с алгоритмом Форда-Фалкерсона обеспечивает высокую практическую скорость на разреженных планарных графах триангуляции Делоне	

Таблица 6: Генерация точек

	Квазислучайная последовательность Соболя [17]	Последовательность Халтона [26]	Диск Пуассона [27]
Описание	Квазислучайная низкорасходящаяся последовательность, основанная на XOR направляющих чисел, построенных по таблице примитивных многочленов над GF(2)	Каждая координата вычисляется как сумма дробей в системе с соответствующим основанием	Генерирует точки с гарантированным минимальным расстоянием между любыми двумя точками. Использует пространственное разбиение для проверки соседей
Воспроизводимость	Поскольку используется скремлирование, то последовательность с очень малой вероятностью окажется воспроизводимой	Полностью детерминирована - одинаковые параметры всегда дают одинаковую последовательность	Воспроизводима при фиксированном seed генератора случайных чисел, используемого внутри алгоритма
Скорость генерации	O(1) на точку - фиксированное число операций XOR над направляющими числами	O(1) на точку - арифметические операции с дробями в заданном основании	O(N) суммарно, однако каждая точка требует проверки соседей в пространственной сетке. Медленнее остальных методов при большом N
Вывод	Последовательность Соболя является оптимальным выбором: сочетает высокую равномерность распределения, исключающую кластеры при триангуляции Делоне, со скоростью O(1) на точку и воспроизводимостью через скремблирование для генерации уникальных уровней		

Оценка вычислительной сложности

Так как триангуляция Делоне является планарным графом, по формуле Эйлера для планарных графов E не превышает $3N - 6$, то есть $E = O(N)$

Сложность вычисления одной точки - $O(1)$. Сложность для N : $O(N)$

Триангуляция Делоне: $O(N \log N)$

Построение графа из ребер триангуляции: $O(E)$

Алгоритм Диницы: $O(N^2 \times E)$

Итоговая временная сложность Алгоритма типа 2 составляет $O(N^2 \times E)$

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.3 Описание и обоснование состава технических и программных средств

3.3.1 Состав технических и программных средств

Для обеспечения безотказной работы приложения мной был выбран состав технических и программных средств, описанный ниже:

1. Язык программирования: C# 9.0 (.NET Standard 2.1)
2. Игровой движок: Unity 2022.3.62f3 LTS
3. Конвейер рендеринга: Universal Render Pipeline (URP) 14.0.12
4. Система авторизации: Firebase Authentication 13.9.0
5. База данных: Firebase Realtime Database 13.9.0
6. Локализация: Unity Localization 1.5.11
7. Сторонняя библиотека: DelaunatorSharp (триангуляция Делоне)
8. Целевая платформа: Android 7.0 и выше
9. Среда разработки: Visual Studio Code
10. Инструмент для создания дизайна: Figma
11. Система контроля и хранения версий: Git/GitHub

3.3.2 Обоснование состава технических и программных средств

1. C# 9.0 (.NET Standard 2.1)

Строго типизированный язык с развитой системой типов и современными возможностями, такими как асинхронное программирование. Обеспечивает высокую производительность и широкую совместимость с платформами, поддерживающими .NET Standard.

2. Unity 2022.3.62f3 LTS

Игровой движок с долгосрочной поддержкой, предоставляющий стабильную среду для разработки игровых приложений. Версия LTS гарантирует отсутствие критических изменений в течение цикла поддержки, что снижает риски при разработке.

3. Firebase Authentication 13.9.0

Инструмент для управления учетными записями, поддерживающее аутентификацию по электронной почте и паролю и обеспечивающее безопасное хранение учетных данных

4. Firebase Realtime Database 13.9.0

Поддерживает синхронизацию состояния между клиентами в реальном времени через WebSocket-соединение, что исключает необходимость периодических запросов к серверу. Обеспечивает офлайн-доступ к данным за счет локального кэширования и автоматической синхронизации при восстановлении соединения.

5. DelaunatorSharp

Производительная реализация триангуляции Делоне для платформы .NET.

Используется для процедурного построения планарных графов при генерации уровней второго типа.

6. Android 7.0 и выше

Целевая платформа, охватывающая подавляющее большинство актуальных устройств на рынке. Версия 7.0 выбрана как нижняя граница совместимости, обеспечивающая доступ к современным API при широком охвате аудитории.

7. Visual Studio Code

Легкий редактор кода с поддержкой расширений для C# и Unity, обеспечивающий подсветку синтаксиса, навигацию по коду.

8. Figma

Браузерный инструмент для проектирования интерфейсов, позволяющий создавать

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

интерактивные прототипы и передавать дизайн-макеты в разработку без установки дополнительного программного обеспечения.

9. Git/GitHub

Система контроля версий и облачный репозиторий, обеспечивающие отслеживание истории изменений, ветвление и резервное хранение исходного кода на всем протяжении разработки.

3.3.3 Описание и обоснование архитектуры программы

- Scripts/UI - уровень пользовательского интерфейса. Здесь реализованы все экраны приложения: главное меню, выбор уровня, окно результатов, таймер, настройки. Компоненты отвечают исключительно за отображение данных и реакцию на действия пользователя.
- Scripts/Firebase - уровень внешних сервисов. Здесь сосредоточено все взаимодействие с Firebase Authentication: регистрация, вход, сброс пароля, отслеживание состояния сессии
- Scripts/ScoreSystem - уровень результатов и базы данных. Тут находится реализация программ, производящих подсчет числа очков по результатам прохождения уровня.
- Scripts/Logic/*/Domains - уровень предметной области. Здесь содержится чистая игровая логика, не зависящая от Unity.
- Scripts/Logic/*/Interfaces - уровень адаптеров и контрактов, связывающий доменную логику со сторонними библиотеками.
- Scripts/Logic/*/View - уровень игрового представления конкретных объектов поля. Папка хранит файлы, позволяющие игре реагировать на действия игрока.
- Scenes - содержит сцены приложения: авторизация, главное меню, выбор уровня и игровые уровни обоих типов.
- Prefabs - библиотека готовых игровых объектов (префабов), задействованных при процедурной генерации уровней и сборке сцен.
- Textures - хранилище графических ресурсов: текстуры, иконки и фоновые изображения, используемые в сценах и интерфейсе.
- Materials - набор материалов URP, определяющих визуальное отображение объектов на сцене
- Audio - директория звуковых ресурсов: фоновая музыка и звуковые эффекты интерфейса
- Fonts - шрифты, применяемые компонентами TextMeshPro во всех сценах приложения.
- Shaders - пользовательские шейдеры на основе Shader Graph, реализующие специальные визуальные эффекты.

3.4 Описание алгоритма работы программы

При запуске приложения выполняется проверка состояния авторизации через Firebase Authentication. Если пользователь не вошел в аккаунт, на весь экран отображается стартовый экран с двумя кнопками: перехода в окно входа и перехода в окно регистрации. При регистрации пользователь вводит имя, электронную почту, пароль и его подтверждение. Клиентская часть проверяет посимвольное совпадение паролей, а также соответствие всех полей ограничениям из таблицы входных данных. При успешной проверке запрос на создание учетной записи передается в Firebase Authentication, после

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

чего профилю назначается отображаемое имя и аватар по умолчанию, а пользователь перенаправляется в Главное Меню. При входе в существующий аккаунт аналогичная валидация выполняется перед отправкой запроса на аутентификацию; в случае неверной комбинации почты и пароля пользователю предлагается повторить ввод или сбросить пароль через письмо на электронную почту.

После авторизации пользователь оказывается в Главном Меню, из которого доступны три раздела: меню выбора уровня, настройки и профиль. Представлены два типа уровней: восстановление связности графа вращением вершин и поиск максимального потока. Над карточкой каждого уровня расположена кнопка просмотра таблицы рекордов, отображающей топ-5 результатов для данного типа уровня.

При нажатии кнопки начала уровня запускается процедурная генерация игрового поля соответствующим алгоритмом. После завершения генерации начинается отсчет времени, а игрок получает возможность взаимодействовать с уровнем

Для уровня типа 1 нажатие на вершину вращает прикрепленные к ней ребра с анимацией поворота. Для уровня типа 2 свайп по соответствующему ребру увеличивает объем потока в нем. В верхней части экрана непрерывно отображается затраченное время. Игрок может поставить игру на паузу кнопкой в правом верхнем углу, после чего появляется диалоговое окно. Игровой процесс продолжается до момента восстановления связности графа для уровня типа 1 или достижения потока максимального объема для уровня типа 2.

По завершении уровня взаимодействие блокируется, воспроизводится анимация завершения и осуществляется переход в окно результатов. Из окна результатов доступны кнопки возврата в Главное Меню и перехода к следующему уровню.

В разделе настроек пользователь регулирует громкость фоновой музыки и звуковых эффектов по шкале от 0 до 10, а также выбирает язык интерфейса - русский или английский.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4 ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1 Ориентировочная экономическая эффективность

В рамках данной работы расчет экономической эффективности не предусматривается.

4.2 Предполагаемая потребность

Ожидается, что разрабатываемая игра будет востребована среди пользователей, предпочитающих короткие казуальные головоломки, а также среди игроков, которым интересны спокойные логические задачи, основанные на визуальном взаимодействии со структурами. Игра может привлечь аудиторию, которая любит медитативный игровой процесс, головоломки без жестких временных ограничений и визуально приятные механики, связанные с ростом и трансформацией объектов.

4.3 Экономические преимущества разработки по сравнению с лучшими отечественными и зарубежными образцами или аналогами

В таблице 7 и 8 представлен сравнительный анализ функциональных аналогов казуальной игры «Цветение» для уровней типа 1 и 2.

«+» ставится, если игра имеет в точности такую же функциональность, что описана в критерии

«-» ставится, если аналог не имеет функциональности или она реализована другим образом

Анализ для уровня типа 1 осуществлялся по следующим критериям:

- Стартовый граф несвязный
- Прохождение уровня только после восстановления связности графа
- Запрет на пересечения ребер
- Процедурная генерация уровней
- Восстановление связности на основе вращения ребер

Анализ для уровня типа 2 осуществлялся по следующим критериям:

- Наличие вершин-истоков и вершин-стоков
- Ребра имеют числовую пропускную способность
- Прохождение уровня при достижении максимального потока
- Несколько истоков и стоков одновременно
- Процедурная генерация уровней

Таблица 7:

Аналог Критерий	Цветение	Sprouts	Hashiwokake ro	Flow Free	Energy
Стартовый граф несвязный	+	-	-	+	+
Прохождение уровня только после восстановления связности графа	+	-	+	-	+

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Запрет на пересечения ребер	+	+	+	+	+
Процедурная генерация уровней	+	+	+	–	–
Восстановление связности на основе вращения ребер	+	–	–	–	+

Таблица 8

Аналог Критерий	Цветение	Flow Free	Plumber	Mini Metro	Liquid Conect
Наличие вершин-истоков и вершин-стоков	+	+	+	+	+
Ребра имеют числовую пропускную способность	+	–	–	+	–
Прохождение уровня при достижении максимального потока	+	–	–	–	–
Несколько источников и стоков одновременно	+	+	–	+	–
Процедурная генерация уровней	+	+	+	+	–

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

18
RU.17701729.05.03-01 ТЗ 01-1
ПРИЛОЖЕНИЕ 1
ТЕРМИНЫ И СОКРАЩЕНИЯ

- Диалоговое окно (Всплывающее диалоговое окно) - информационное окно, возникающее в центре экрана
- Модификация - изменение в структуре графа
- Скремблирование - обратимое преобразование цифрового потока или аналогового сигнала для получения псевдослучайной структуры без изменения скорости передачи
- Детерминированность - свойство системы или процесса выдавать одинаковый результат при одинаковых входных данных и начальных условиях.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ССЫЛКИ НА ПРИЛОЖЕНИЯ АНАЛОГИ

- Hashiwokakero
<https://www.puzzle-bridges.com/>
- Flow Free
<https://play.google.com/store/apps/details?id=com.bigduckgames.flow&hl=ru>
- Energy
https://poki.com/ru/g/energy#utm_source=redirect-en-ru
- Plumber
<https://www.crazygames.com/game/plumber>
- Mini Metro
<https://play.google.com/store/apps/details?id=nz.co.codepoint.minimetro>
- Liquid Connect
<https://apps.apple.com/us/app/liquid-connect/id6757105766>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.05.03-01 ТЗ 01-1
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77. Виды программ и программных документов. // Единая система программной документации. -М.: Стандартиформ, 2005.
2. ГОСТ 19.102-77. Стадии разработки. // Единая система программной документации. -М.: Стандартиформ, 2005.
3. ГОСТ 19.103-77. Обозначения программ и программных документов. // Единая система программной документации. -М.: Стандартиформ, 2005.
4. ГОСТ 19.104-78. Основные надписи. // Единая система программной документации. -М.: Стандартиформ, 2005.
5. ГОСТ 19.105-78. Общие требования к программным документам. // Единая система программной документации. -М.: Стандартиформ, 2005.
6. ГОСТ 19.106-78. Требования к программным документам, выполненным печатным способом. // Единая система программной документации. -М.: Стандартиформ, 2005.
7. ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. -М.: Стандартиформ, 2005.
8. ГОСТ 19.602-78: Правила дублирования, учета и хранения программных документов, выполненных печатным способом. //Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.603-78. Общие правила внесения изменений. // Единая система программной документации. -М.: Стандартиформ, 2005.
10. ГОСТ 19.604-78. Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. - М.: Стандартиформ, 2005.
11. ГОСТ 19.301-79. Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. -М.: Стандартиформ, 2005.
12. ГОСТ 19.401-78. Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. -М.: Стандартиформ, 2005.
13. ГОСТ 19.404-79. Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. -М.: Стандартиформ, 2005.
14. ГОСТ 19.505-78. Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. -М.: Стандартиформ, 2005.
15. Buchin K., Schulz A. On the number of spanning trees a planar graph can have // arXiv.org. - 2010. - arXiv:0912.0712 [math.CO]. - URL: <https://arxiv.org/abs/0912.0712> (дата обращения: 08.05.2026)
16. Триангуляция Делоне // neerc.ifmo.ru. URL: https://neerc.ifmo.ru/wiki/index.php?title=Триангуляция_Делоне (дата обращения: 08.05.2026).
17. Sobol sequence // Wikipedia. URL: https://en.wikipedia.org/wiki/Sobol_sequence (дата обращения: 08.05.2026).
18. Алгоритм Прима // neerc.ifmo.ru. URL: http://neerc.ifmo.ru/wiki/index.php?title=Алгоритм_Прима (дата обращения: 08.05.2026).
19. Алгоритм Краскала // Википедия. URL: https://ru.wikipedia.org/wiki/Алгоритм_Краскала (дата обращения: 08.05.2026).
20. RandomNumberGenerator Class // Microsoft Learn. URL: <https://learn.microsoft.com/ru-ru/>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- ru/dotnet/api/system.security.cryptography.randomnumbergenerator?view=net-8.0 (дата обращения: 08.05.2026).
21. Random Class // Microsoft Learn. URL: <https://learn.microsoft.com/ru-ru/dotnet/api/system.random?view=net-10.0> (дата обращения: 08.05.2026).
 22. Граф Габриэля // Википедия. URL: https://ru.wikipedia.org/wiki/Граф_Габриэля (дата обращения: 08.05.2026).
 23. Метод ближайших соседей // neerc.ifmo.ru. URL: https://neerc.ifmo.ru/wiki/index.php?title=Метрический_классификатор_и_метод_ближайших_соседей (дата обращения: 08.05.2026).
 24. Алгоритм Диницы // neerc.ifmo.ru. URL: https://neerc.ifmo.ru/wiki/index.php?title=Схема_алгоритма_Диница (дата обращения: 08.05.2026)
 25. Алгоритм Форда-Фалкерсона, реализация с помощью поиска в глубину // neerc.ifmo.ru. URL: https://neerc.ifmo.ru/wiki/index.php?title=Алгоритм_Форда-Фалкерсона,_реализация_с_помощью_поиска_в_глубину (дата обращения: 08.05.2026)
 26. Halton sequence // Wikipedia. URL: https://en.wikipedia.org/wiki/Halton_sequence (дата обращения: 08.05.2026)
 27. Poisson disk sampling through disk packing // ResearchGate. URL: https://www.researchgate.net/publication/282599935_Poisson_disk_sampling_through_disk_packing (дата обращения: 08.05.2026)
 28. Depth-first search // Wikipedia. URL: https://en.wikipedia.org/wiki/Depth-first_search (дата обращения: 08.05.2026)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Номера листов (страниц)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.03-01 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата